Week 3 — Your Agent Team

Mission. Tuesday you rebuild your desk as an agent team whose process is a graph in code: the same nodes in the same order on every run, each role a model call on what the graph hands it, and a verifier after the Manager that no run can skip. Thursday each role's work becomes yours: a persona you wrote, which must draw an insight, proved on a company your partner picks. Teams form.

Why this matters

In 2005 a freestyle chess tournament let anyone enter with any help. The winners were two amateurs running three ordinary computers at once, ahead of grandmasters with stronger machines. Kasparov wrote: "Weak human + machine + better process was superior to a strong computer alone and, more remarkably, superior to a strong human + machine + inferior process." Every desk in this room calls the same models. What you own is the process: who runs, in what order, on what inputs, with what check at the end. The paper's team let a model decide that. Yours will not.

You arrive with

Your desk from Lab 03 with every number sourced, two audited reports, Checkpoint 1 filed, and your one line on who decides which role runs next in your desk: code, or a model.

Outcomes

- Rebuild the four roles as an agent team in a framework you chose: the process a graph in code, each role a node that receives only what the graph sends it, every run saved.
- Make the verifier a node after the Manager that runs on every run, and prove it on a seeded error, an invented number and a clean report.
- Write a persona for each analyst role, in your own words, that must state one insight the release's headline does not.
- Say which decisions in your agent team are code's and which are the model's.

- Form a team.

The sessions

- **Session 4 —** Lab 04: The agent team.
- **Session 5 —** Lab 05: Make the agent team yours. Checkpoint 2 is due by the end of the day; teams form.

Read this week

Required, two:

- **Kasparov, G. (2010).** *The Chess Master and the Computer.* The New York Review of Books. nybooks.com. About 20 minutes. Write one line: in your Lab 03 desk, what is the process?
- **Your framework's first page**, one of four, about 20 minutes, before Tuesday: LangGraph, the OpenAI Agents SDK (its financial research example runs a verifier agent last), PydanticAI, or CrewAI, the paper's framework, whose Flows fix the order in code. Each page says which Python versions it takes; pick one that takes yours. The Lab 04 request names your choice. The list is checked against the frameworks' current releases before each offering; a framework that stops taking current Python leaves it.

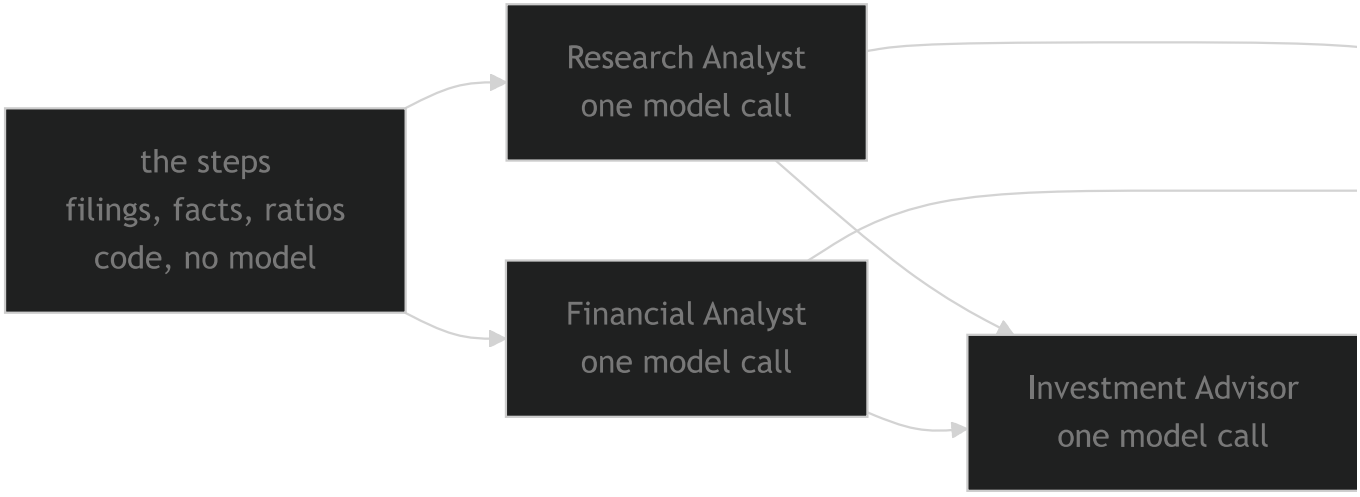
Optional, two:

- **Anthropic (2024),** *Building effective agents* (the Week 1 reading), the two sections "Workflow: orchestrator-workers" and "Workflow: evaluator-optimizer". About 10 minutes. Which one is your Manager, and which one is your Verifier?
- **Francis & Zhang (2026),** Section 3.1, once more: the paper's team is hierarchical, and a manager agent coordinates it. Write one line: what does a manager that coordinates decide that a graph would decide for you?

Teaching 1 — process is the thing to own

The paper's team is hierarchical: a manager agent decides which role to call, when, and with what. That is a model deciding the process. It is also why the paper cannot say, for any report, why a number is there: the path that produced it could differ from one run to the next. Every framework on the list offers the other way, a fixed order in code. Yours is a graph: a state,

nodes, edges. The nodes, in order: the steps from Lab 02; the Research Analyst; the Financial Analyst; the Investment Advisor; the Strategy Manager; the Verifier. The edges are code. Run it twice on the same ticker and the same nodes run in the same order. The one retry the Manager may send is a slot in the graph, not a choice a model makes. What differs is what each role writes.



The week's rule: **the process is deterministic; the work inside each node is dynamic.** Code decides who runs, what each node sees, what comes back for a second try, and what is saved. The model decides what the business is, which development is the catalyst, what the thesis is, and what the one insight is. An agent team that lets the model decide the first list cannot be audited: nobody can say why a node ran. An agent team that lets code fill the second list has nothing to say: a template with numbers in it is not analysis.

Code decides	**The model decides**
which node runs next, and when the run stops	what the business is and where it stands
what each node is shown, as JSON: task, context, expected output	which development is the catalyst
that a part missing from the template goes back to its role, once	the thesis, and the one number that decides it
that the Verifier runs after the Manager, on every run	whether a line states that number, for that period, in that definition
what is saved, and where	the one insight the headline does not state

Teaching 2 — the verifier as a node

One run produces a report and saves everything it read: the filing texts and the XBRL facts. The Verifier is the next node. It receives only those and the report. It never sees the other roles' instructions and never reruns them. For every sourced number in the report it does two checks. In code: a number sourced to an XBRL tag and period is compared with the facts file exactly; a number sourced to a line is found in the filing text, allowing for spacing; a ratio is recomputed from its quoted inputs. With the model: does that line state that number, for that period, in that definition? Most of the Financial Analyst's part is checked in code alone, which is why the run saves the facts it read.

Worked example, the shape, not a figure. A report states diluted EPS one dollar above the 10-K's per-share line and sources it to that line. The code check fails: the facts file holds the 10-K's figure, and no line in the 10-K says the report's. Marked wrong, corrected, the real line beside it. A report states a revenue growth rate and quotes its two inputs, but the inputs give a different rate: marked wrong, corrected. A report states a return on equity with no definition, where the filing gives both average and year-end equity: unsupported, because the report did not say which. Lab 04 seeds each of these into a copy of your own run.

Two things a verifier cannot catch by design. A report that sources a true line for a claim the client did not ask about, in place of the one they did: the right number, the wrong claim, and that is judgment, the audit's "missing" mark. And the Investment Advisor's thesis, which no filing can confirm. Say both out loud when you present your team.

Teaching 3 — the persona: dynamic work in a fixed slot

A persona is a role's instruction file that fixes what the role values, what it is given and what it must return. The model fills it on the filings of the day, differently each run. The shape, adapted from the expert template in the instructor's FAskills, six parts:

1. **Identity.** Who the role is, in two sentences.
2. **How it reads a company.** What it looks at first, and what it ignores.
3. **What it is given.** Which steps' output, named; nothing else.
4. **The criteria it scores.** Each with a number and the line or tag it came from.
5. **What would change its view.** The one thing in the next filing that would.
6. **The output.** The part of the template it owns, ending with one line that begins "Insight:" and states a claim the filings support and the release's first paragraph does

not, with the line it came from.

The verifier checks the number in the insight line like any other. Whether the insight matters is for your partner, and later your client, to judge. You write the personas yourself, on Thursday, in your own words: they are the part of the desk that is yours.

Better than the paper's team

- The process is in code, drawn as a graph, the same on every run; the paper's manager model decided it.
- A verifier node the paper did not have, after the Manager, with no flag that turns it off.
- Every role sees only the state the graph sends it; every run saves what every node read and wrote.
- The tools are your Week 2 steps, in code, so every number carries its line; the paper's web search is optional.
- Each role must draw an insight, and the Manager refuses a part that only restates the headline.
- The run stops: the Manager sends a part back once, no role runs more than twice, and the run says so. The tokens are yours.

Three levels, from this week on

- **Must.** The agent team with the verifier as a node no run can skip, three personas with insight lines, proved on your partner's pick. Checkpoint 2 on the grading page.
- **Expand.** A second model as a second verifier node, its marks compared with the first's; peers chosen by the Financial Analyst's own criteria; a run that stops itself when a role's output is empty, with the reason saved.
- **Ship.** Your team's repository has a README that states the command, the graph drawn, and a first release tag. Every week from here names one move toward a product someone else could run.

Teams

Three students. Form by the end of Session 5 (the schedule); the instructor places anyone unplaced then. From Week 4 the team's desk writes the live record. How you make one team

out of three, and where you keep it, is yours to decide.

Before Session 4 — about 60 minutes

1. Push everything from Lab 03 to your repository (never `.env`), and audit all eleven core claims of your Lab 03 report yourself, as that lab says.
2. **Fix the role that erred.** For each error in your audit, send your agent: "In my desk, the [role name] produced [the wrong number] where the filing says [the right number] at [the line or tag]. Change that role's instructions so that every number it states is followed by the filing and the exact line or tag it came from, quoted, and so that it writes 'not stated in the filings' rather than estimating. A number computed from quoted numbers is allowed when the arithmetic is shown. Do not change the other roles." No errors? Send the same request for the Financial Analyst without the first sentence: Tuesday's verifier needs those sources. Rerun on COST: every number now carries its source.
3. Read Kasparov. Then open your Manager's code and write one line: who decides which role runs next, code or a model?
4. Pick a framework from the four above and read its first page. Write its name into the bracket of the Lab 04 request before Tuesday.

Before Session 5 — about 45 minutes

1. Push the agent team to your repository (never `.env`). Finish the seeded-error, invented-number and clean-report tests if the room ran out. Checkpoint 2 needs the seeded-error marks (the grading page); the other two are yours to keep.
2. Draft the three personas, in the six parts of Teaching 3, about ten minutes each. On paper or in a file; no AI writes them.
3. Talk to two classmates about a team.

Next: in Week 4 the desk serves its first client, an investor, and a book's chapter becomes a skill your agent runs: the tests in code, the judgment in the model, a hook that checks the numbers. Your own code brings the price.